

Formulário de Projeto — Fase 2

NutriTrack — Sistema de Gerenciamento de Consumo Nutricional

Componentes do Grupo:

Pedro Augusto Silva Ferreira, Luis Felipe Almeida Rodrigues

Pontifícia Universidade Católica de Minas Gerais — ICEI

Algoritmos e Estruturas de Dados III — 2026

Respostas Completas ao Formulário

a) Estrutura usada para representar os registros

Cada registro implementa a interface `app.Registro`, que define os métodos `getId()`, `setId(int)`, `toByteArray()` e `fromByteArray(byte[])`. A serialização usa `DataOutputStream` sobre `ByteArrayOutputStream`, gerando um `byte[]` de tamanho variável por registro.

No arquivo `.db`, cada registro é gravado no formato:

```
[lápide : 1 byte] [tamanho : short / 2 bytes] [dados :  
byte[tamanho]]
```

antecedido pelo cabeçalho global do arquivo:

```
[últimoId : int / 4 bytes] [ponteiroListaRemovidos : long / 8  
bytes]
```

As classes de domínio (`Usuario`, `Alimento`, `Refeicao`, `Consumo`) escrevem seus campos em ordem fixa: tipos primitivos via `writeInt/writeFloat/writeLong`, strings via `writeUTF` e datas via `writeLong(epochDay)`.

b) Como atributos multivalorados do tipo string foram tratados

As listas de strings (`Usuario.telefones` e `Alimento.tags`) são serializadas em sequência usando um cabeçalho de quantidade:

```
[short qtd] [writeUTF s_1] [writeUTF s_2] ... [writeUTF s_qtd]
```

Na leitura, o método `fromByteArray` chama `readShort()` para obter a quantidade e itera chamando `readUTF()`, recompondo o `List<String>` original. Isso permite tamanho variável sem desperdício de espaço.

c) Como foi implementada a exclusão lógica

Cada registro carrega 1 byte de **lápide**: `' '` (0x20) indica ativo e `'*'` (0x2A) indica removido. A operação `delete(id)` faz:

1. Localiza o endereço do registro pelo índice primário (Hash Extensível) — O(1) médio.
2. Sobrescreve o byte da lápide com `'*'`.
3. Insere o endereço na **lista encadeada de espaços livres**, ordenada por tamanho (best-fit).
4. Remove a entrada (`id` → endereço) do índice primário.
5. Se for um `Consumo`, remove também a entrada (`refeicaoId` → `consumoId`) do índice 1:N.

Em uma inserção/atualização que precise de mais espaço, o método `getDeleted(tam)` percorre a lista buscando o primeiro nó com tamanho suficiente. Caso não encontre, o registro é gravado no fim do arquivo.

d) Além das PKs, quais outras chaves foram utilizadas

Além das chaves primárias auto-incrementadas, o sistema utiliza três chaves estrangeiras:

- **FK** `Refeicao.usuarioId` → `Usuario.id` — validada na inserção/atualização.
- **FK** `Consumo.refeicaoId` → `Refeicao.id` — usada como chave do índice 1:N (Refeição → Consumos).
- **FK** `Consumo.alimentoId` → `Alimento.id` — validada para integridade referencial.

A chave `refeicaoId` é especialmente importante porque, além de validada como FK, serve como **chave de busca** no índice 1:N que retorna todos os consumos de uma refeição.

e) Como a estrutura (hash) foi implementada para cada chave de pesquisa

A classe `HashExtensivel<T extends RegistroHashExtensivel<T>>` é totalmente genérica. Cada índice é instanciado com um número fixo de slots por cesto e um `Constructor<T>` para reconstruir entradas a partir de bytes.

O hash usa os *profundidadeGlobal* bits menos significativos de `hashCode()` para indexar o diretório: `idx = hashCode & ((1 << profGlobal) - 1)`. Quando um cesto enche, ocorre **split**: a profundidade local é incrementada, um novo cesto é alocado no fim do arquivo de cestos, os ponteiros do diretório são reescritos e os elementos são redistribuídos. Se a profundidade local ultrapassa a global, o diretório é dobrado.

O método `read(chave)` retorna **todos** os elementos do cesto cuja chave bate com a busca — fundamental para o índice 1:N onde várias entradas compartilham a mesma FK.

Índice	Tipo de entrada	Chave (hashCode)	Slots/cesto
Primário (cada uma das 4 tabelas)	ParIDEndereco (12 B)	id	8
1:N — Refeicao → Consumo	ParIDID (8 B)	refeicaoId	16

f) Como foi implementado o relacionamento 1:N

O relacionamento **Refeição → Consumos** é implementado por um Hash Extensível dedicado (`consumo_por_refeicao`) cujas entradas são pares `ParIDID(FK = refeicaoId, PK = consumoId)`. Para cada consumo criado, o `ConsumoDAO` insere uma entrada nesse índice.

A consulta `ConsumoDAO.listarPorRefeicao(refId)` executa:

1. `HashExtensivel<ParIDID>.read(refId)` retorna todos os pares (`refId`, `consumoId`).
2. Para cada `consumoId`, chama `Arquivo<Consumo>.read(consumoId)`, que usa o índice primário do arquivo de consumos (também O(1) médio).
3. Devolve a `List<Consumo>`.

A integridade referencial é mantida em três pontos:

- **Criação:** o controller verifica via `exists(id)` que a refeição e o alimento existem antes de gravar o consumo.
- **Alteração de FK:** ao atualizar a `refeicaoId` de um consumo, o DAO remove o par antigo e insere o novo no índice 1:N.

- **Remoção em cascata:** ao remover uma refeição, o `RefeicaoController` consulta o índice 1:N e remove todos os consumos associados (e suas entradas no índice).

g) Como os índices são persistidos em disco

Cada índice é composto por **dois arquivos binários**:

Arquivo	Conteúdo
<code>{nome}.idx.d.db</code>	Diretório: <code>int profundidadeGlobal + 2^profGlobal</code> ponteiros <code>long</code> para cestos.
<code>{nome}.idx.c.db</code>	Cestos: sequência de cestos de tamanho fixo. Cada cesto = <code>byte profundidadeLocal + short qtd + vetor de slots</code> .

Tudo é gravado via `RandomAccessFile`. Cada operação executa `seek + write` imediatamente, sem cache em memória. Os índices são atualizados **na mesma operação** que altera o arquivo de dados (inserção, atualização que muda endereço, remoção). O servidor fecha todos os `RandomAccessFile` via *shutdown hook*, garantindo o flush.

Existe uma rotina de **reconstrução automática**: se o arquivo de índice está vazio mas o `.db` de dados já contém registros, o índice é reconstruído na inicialização varrendo o arquivo de dados — basta apagar a pasta de índices para forçar a reconstrução.

h) Estrutura do projeto no GitHub

O repositório está disponível em <https://github.com/taurusluis2004/TP-AEDS-III> e segue o padrão MVC + DAO:

```

TP-AEDS-III/
├── README.md                    # instruções de compilação
e execução
├── dados/                      # arquivos binários (.db)
+ índices (.idx.*)
|   ├── usuario/               (usuario.db + índice primário)
|   ├── alimento/             (alimento.db + índice primário)
|   ├── refeicao/              (refeicao.db + índice primário)
|   ├── consumo/              (consumo.db + índice primário)
|   └── consumo_por_refeicao/    # índice 1:N (FK
refeicaoId → PK consumoId)
├── src/app/
|   ├── Main.java              # demo guiado (console)
|   ├── ConsoleApp.java        # CRUD via menu (console)
|   ├── Servidor.java          # servidor HTTP embutido
(porta 8080) + REST
|   ├── Registro.java          # interface comum
|   └── Arquivo.java           # persistência genérica +
integração com índice primário
|   ├── model/
|   |   ├── Usuario.java
|   |   ├── Alimento.java
|   |   ├── Refeicao.java
|   |   └── Consumo.java
|   ├── controller/
|   |   ├── UsuarioController.java
|   |   ├── AlimentoController.java
|   |   ├── RefeicaoController.java # remoção em cascata
|   |   └── ConsumoController.java # validação de FKs
|   └── dao/
|       ├── RegistroHashExtensivel.java # interface
|       ├── HashExtensivel.java         # estrutura genérica
|       └── ParIDEndereco.java          # entrada do índice
primário
|           ├── ParIDID.java            # entrada do índice
1:N
|           ├── UsuarioDAO.java
|           ├── AlimentoDAO.java
|           ├── RefeicaoDAO.java
|           └── ConsumoDAO.java         # mantém o índice 1:N
└── web/
    ├── index.html, app.js, styles.css # SPA que consome a
API REST
    └── documentacao.html              # documentação
completa

```

O `README.md` traz instruções para compilar com `javac -d out` e executar tanto o servidor (`java -cp out app.Servidor`) quanto o console (`java -cp out app.ConsoleApp`).